

Классический анализ движения

от одного уравнения — к RAFT

Александр Савельев

Курс: Обработка видео в контексте машинного обучения и ИИ
Лекция 2

Сюжет лекции, не оглавление

Одна нить на все слайды: *одно уравнение на два неизвестных* — каждый метод добавляет второе допущение.

1. Зачем поток и почему его нельзя измерить
2. Постоянство яркости → уравнение OFC
3. Апертура: из одного пикселя видна только часть движения
4. Lucas-Kanade: второе уравнение из *окна*
5. Horn-Schunck и TV: второе уравнение из *соседей*
6. Пирамида: большой сдвиг снова делают маленьким
7. Фон: поток уже не цель, а инструмент
8. Те же три кирпича — слоями сети (FlowNet → RAFT)

1. Что такое оптический поток

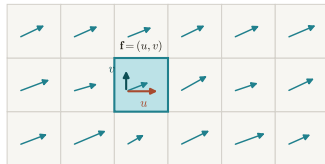
В лекции 1 — блочный поток кодека. Сегодня — сдвиг каждого пикселя.

Определение

Поле $\mathbf{f}(x, y) = (u, v)$.

u — сдвиг вправо за кадр (px),

v — сдвиг вниз (px).



каждый пиксель — свой сдвиг за кадр, в пикселях

Где нужен: кодеки, трекинг, стабилизация, two-stream, интерполяция, VSR, KYC.

Почему нельзя измерить: камера пишет только яркость $I(x, y, t)$. Сдвиг в файле не записан.

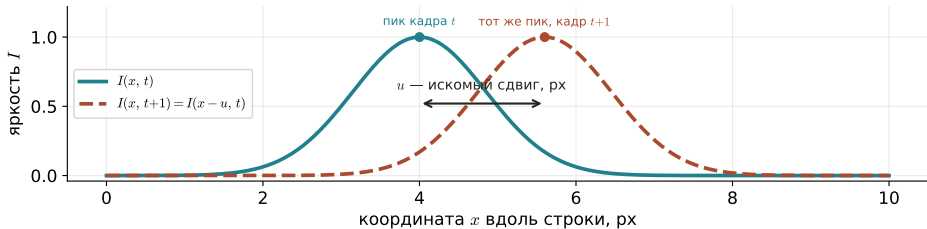
Чтобы оценить (u, v) , нужна модель того, что сохраняется при движении. Это следующая тема — постоянство яркости.

2. Сдвиг в файле не записан — предполагаем, что яркость та же

Гипотеза постоянства яркости

$$I(x + u, y + v, t + 1) = I(x, y, t).$$

I — яркость (обычно канал Y), (x, y) — пиксель в кадре t , (u, v) — искомый сдвиг.



Это модель, не закон физики: ломается на бликах, тенях, окклюзиях, смене света.

2. Подставляем гипотезу в Тейлора — получаем OFC

$$I_x = \partial I / \partial x, I_y = \partial I / \partial y, I_t = \partial I / \partial t.$$

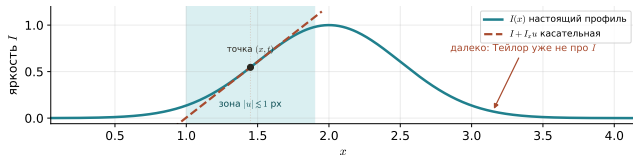
$$I(x + u, y + v, t + 1) \approx I + I_x u + I_y v + I_t.$$

Приравниваем к 0:

OFC — Optical Flow Constraint

$$I_x u + I_y v + I_t = 0.$$

I_x, I_y, I_t известны из картинки; (u, v) — нет.

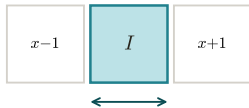


Линеаризация честна при $|f| \lesssim 1-2$ px. Даже тогда уравнений меньше, чем неизвестных.

2. Откуда в OFC берутся I_x, I_y, I_t

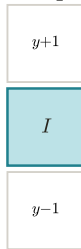
Сначала сглаживаем Гауссом ($\sigma \approx 1-1.5$ px), иначе шум убьёт разности.
Потом — соседние пиксели:

$$I_x \approx \frac{I(x+1, y) - I(x-1, y)}{2}$$



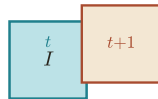
$I_x > 0$: справа светлее, чем слева

$$I_y \approx \frac{I(x, y+1) - I(x, y-1)}{2}$$



$I_y > 0$: снизу светлее, чем сверху

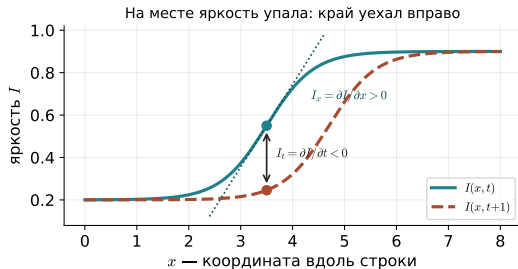
$$I_t \approx I(x, y, t+1) - I(x, y, t)$$



$I_t < 0$: на месте стало темнее

Из одного пикселя приходит *одно* число I_t и *два* неизвестных u, v . Как это выглядит геометрически — следующий слайд.

2. В 1D неизвестная одна, в 2D — уже прямая



В 1D неизвестная одна: $I_x u + I_t = 0$

I_x — наклон профиля (известно)
насколько светлее справа

I_t — падение яркости на месте (известно)
кадр $t+1$ минус кадр t

$u = -I_t / I_x$ — единственный сдвиг
здесь $u = 1.2$ px

В кадре то же самое, но неизвестных две: $I_x u + I_y v = -I_t$ задаёт *прямую* в плоскости (u, v) , не точку. Это апертурная проблема.

3. Из одного пикселя видна только часть движения

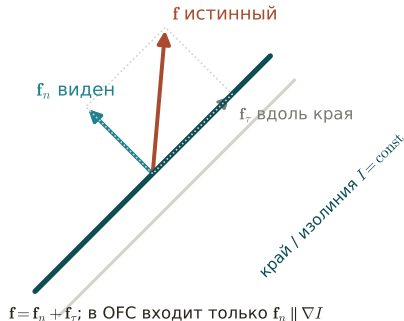
Край в круглой дырке: видна только составляющая *перпендикулярно* краю. Вдоль края яркость не меняется.

Нормальный поток $\mathbf{f}_n$

$$\mathbf{f} = \mathbf{f}_n + \mathbf{f}_\tau,$$

$$\mathbf{f}_n = -\frac{I_t}{\|\nabla I\|^2} \nabla I.$$

$\nabla I = (I_x, I_y)$. $\mathbf{f}_\tau$ в OFC не входит.

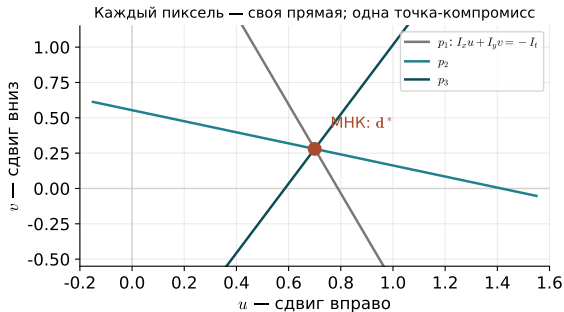


4. Lucas-Kanade: второе уравнение берём из окна

Допущение LK (1981)

Все n пикселей окна W ($5 \times 5 \dots 21 \times 21$) имеют один вектор $\mathbf{d} = (u, v)$.
Вместо 1 уравнения на 2 неизвестных — n на 2.

Допущение LK: одно окно — один вектор



4. Система переопределена — решаем МНК

Для каждого $p_i \in W$: $I_x(p_i) u + I_y(p_i) v = -I_t(p_i)$.

$$A\mathbf{d} = -\mathbf{b}, \quad A = \begin{bmatrix} I_x(p_1) & I_y(p_1) \\ \vdots & \vdots \\ I_x(p_n) & I_y(p_n) \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} I_t(p_1) \\ \vdots \\ I_t(p_n) \end{bmatrix}, \quad \mathbf{d} = \begin{bmatrix} u \\ v \end{bmatrix}.$$

A — $n \times 2$: строки есть градиенты пикселей окна. Из-за шума система несовместна $\Rightarrow$ минимум $\|A\mathbf{d} + \mathbf{b}\|_2^2$:

$$\mathbf{d} = -(A^T A)^{-1} A^T \mathbf{b}.$$

Обращать нужно матрицу $G = A^T A$ размера 2×2 . Когда это можно — следующий слайд.

4. Вся разрешимость сидит в структурном тензоре G

Структурный тензор

$$G = A^T W A = \begin{bmatrix} a & b \\ b & c \end{bmatrix}, \quad \begin{aligned} a &= \sum w l_x^2, \\ b &= \sum w l_x l_y, \\ c &= \sum w l_y^2. \end{aligned}$$

$w(p)$ — гауссовы веса: центр окна важнее края.

G симметричная и неотрицательная. Её собственные значения $\lambda_1 \geq \lambda_2 \geq 0$ скажут, есть ли в окне две независимые ориентации градиента.

Чтобы читать G , нужно помнить, что такое λ у матрицы 2×2 . Сейчас — короткое повторение, сразу вернёмся к углу и краю.

4. Мини-повторение: что такое λ

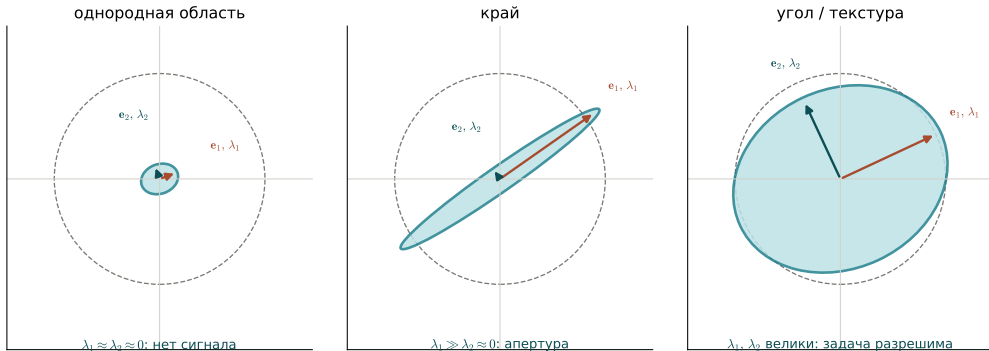
Только что получили $G = A^T A$. Когда её можно обратить?

Собственный вектор

$G\mathbf{e} = \lambda\mathbf{e}$: направление $\mathbf{e}$ матрица не поворачивает, а растягивает в λ раз. У симметричной G векторы $\mathbf{e}_1 \perp \mathbf{e}_2$, $\lambda_1 \geq \lambda_2 \geq 0$.

Геометрия: G превращает единичную окружность в эллипс. Оси эллипса — $\mathbf{e}_1, \mathbf{e}_2$, длины — λ_1, λ_2 .

4. Эллипс неопределённости — три случая на одном рисунке



Через слайд то же самое на патчах изображения: на углу эллипс круглый, на крае — палка.

4. Формула λ для матрицы 2×2

$$G = \begin{bmatrix} a & b \\ b & c \end{bmatrix}, \quad \lambda_{1,2} = \frac{1}{2} \left(a + c \pm \sqrt{(a - c)^2 + 4b^2} \right).$$

Без диагонализации: $\text{tr}(G) = a + c = \lambda_1 + \lambda_2$, $\det(G) = ac - b^2 = \lambda_1 \lambda_2$.

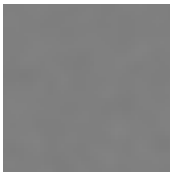
Окно	λ_1	λ_2	LK?
однородное	≈ 0	≈ 0	нет сигнала
край	большое	≈ 0	апертура
угол / текстура	большое	большое	да

$\det G = 0$ — апертура в цифрах. Сейчас посчитаем на угле и на крае.

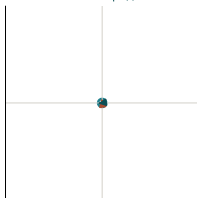
4. Те же три случая — на окне изображения

Эллипс собственных значений матрицы $A^T A$ (структурного тензора) в окне 41×41

однородная область

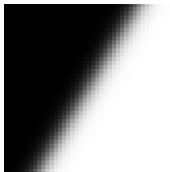


$\lambda_1 \approx \lambda_2 \approx 0$:
поток не определён

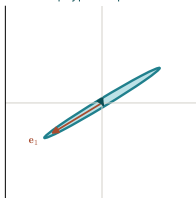


$\lambda_1 = 0.0, \lambda_2 = 0.00$

край



$\lambda_1 \gg \lambda_2 \approx 0$:
апертурная проблема



$\lambda_1 = 6.0, \lambda_2 = 0.00$

угол / текстура



$\lambda_1, \lambda_2 \gg 0$:
система хорошо обусловлена



$\lambda_1 = 2.3, \lambda_2 = 1.80$

4. Считаем по формуле: угол, оба λ велики

Окно 3×3 , истинный сдвиг $(u, v) = (1.2, 0.6)$. Конечные разности:

$$I_x = \begin{bmatrix} 10.5 & 3 & -4.5 \\ 10.5 & 3 & -4.5 \\ 10.5 & 3 & -4.5 \end{bmatrix} \quad I_y = \begin{bmatrix} 9.5 & 9.5 & 9.5 \\ 2 & 2 & 2 \\ -5.5 & -5.5 & -5.5 \end{bmatrix} \quad I_t = \begin{bmatrix} -20.3 & -14.9 & -13.9 \\ -21.2 & -6.1 & -3.2 \\ -21.2 & -6.5 & -3.7 \end{bmatrix}$$

$$G = \begin{pmatrix} 418.5 & 54 \\ 54 & 373.5 \end{pmatrix}, \quad \lambda_2 = 337.5, \lambda_1 = 454.5 \text{ — оба большие.}$$

$\mathbf{d}_{\text{оц}} = (1.45, 0.74)$ против истины $(1.20, 0.60)$. Зазор — Тейлор при сдвиге > 1 пх и грубые разности.

4. Тот же код на чистом крае: Singular matrix

Вертикальный край, сдвиг по диагонали. Вдоль края $I_y \equiv 0$:

$$G = \begin{pmatrix} 600 & 0 \\ 0 & 0 \end{pmatrix}, \quad \lambda = (600, 0), \quad \det G = 0.$$

u восстановить можно, v — любой. Это арифметика слайда про дырку.

Поэтому точки для трекинга — места, где оба λ велики. Это не новый объект: та же G , другой вопрос. Дальше — Харрис и Shi-Tomasi.

4. Листинг: Lucas-Kanade на одном окне

Разбор закрыт. Весь метод — один проход по окну.

Алгоритм: Lucas-Kanade

вход: I_1, I_2 ; центр p ; окно W ($5 \times 5 \dots 21 \times 21$); веса w

выход: $\mathbf{d} = (u, v)$ или отказ

1. сгладить оба кадра Гауссом ($\sigma \approx 1-1.5$)
2. в окне W посчитать I_x, I_y, I_t
3. собрать $G = \sum w \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$, $\mathbf{b} = \sum w I_t \begin{bmatrix} I_x \\ I_y \end{bmatrix}$
4. если $\lambda_2(G) \approx 0$ — отказ (нет сигнала / апертура)
5. иначе $\mathbf{d} = -G^{-1} \mathbf{b}$

Это один уровень, одна точка. KLT ниже обернёт это циклом по точкам, итерациями и пирамидой.

5. Харрис: та же G , вопрос «угол или край?»

Отклик Harris (1988)

$$R = \det(G) - k \operatorname{tr}(G)^2 = \lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)^2, \quad k \approx 0.04-0.06.$$

k штрафует край: одно λ большое, другое нуль.



5. Shi-Tomasi выбирает точки, KLT их ведёт

Good Features to Track (1994)

$\min(\lambda_1, \lambda_2) > \tau$ — даже слабая ось G достаточно велика. Это `cv2.goodFeaturesToTrack`. Дальше ведём *эти* точки.

Один шаг LK честен только при $|\mathbf{d}| \lesssim 1-2$ пх. Поэтому KLT не решает большой сдвиг сразу, а крутит Ньютона-Гаусса:

1. есть текущая оценка $\mathbf{d}^{(k)} = (u^{(k)}, v^{(k)})$, старт $\mathbf{d}^{(0)} = \mathbf{0}$
2. **warp** — сдвинуть *второй* кадр на этот $\mathbf{d}$, чтобы пятно снова оказалось «на месте»:

$$I'_2(x, y) = I_2(x + u^{(k)}, y + v^{(k)}).$$

3. на паре (I_1, I'_2) один шаг LK даёт маленькую поправку $\Delta \mathbf{d}$
4. $\mathbf{d}^{(k+1)} = \mathbf{d}^{(k)} + \Delta \mathbf{d}$; пока $\Delta \mathbf{d}$ не станет крошечной

5. Warp: вычитаем уже найденный сдвиг

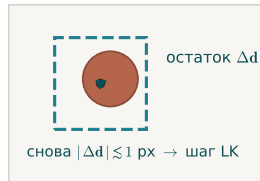
кадр I_1 : точка p и окно W



кадр I_2 : пятно уехало на d



warp: $I'_2(x) = I_2(x + d)$



После варпа в окне остаётся $\lesssim 1 \text{ px}$ — снова зона Тейлора. Следующий слайд — все буквы; потом весь цикл одним листингом.

5. Буквы в листинге KLT

Буква	Что это
W	окно вокруг точки. OpenCV: <code>winSize</code> , часто 15×15 .
L	сколько раз кадр уменьшили вдвое. OpenCV: <code>maxLevel</code> . На грубом уровне большой сдвиг снова маленький.
$\text{warp}(I_2, \mathbf{d})$	$I'_2(x) = I_2(x + \mathbf{d})$: картинку сдвинули на текущий $\mathbf{d}$.
$\Delta \mathbf{d}$	один шаг LK на уже сваренной паре (I_1, I'_2) .
ε	стоп, когда $\ \Delta \mathbf{d}\ < \varepsilon$ (типично 0.01–0.03 px: дальше нет смысла).
$K_{\max}$	потолок итераций <i>на одном уровне</i> . Иначе на шуме цикл не остановится. OpenCV: <code>criteria.maxCount</code> , часто 10–30.
e_{FB}	прогнали трек туда и обратно; если точка не вернулась ближе 1 px — выкинуть.

Sparse: 100–2000 точек, не каждый пиксель.

5. Листинг: KLT целиком

Алгоритм: KLT (Kanade-Lucas-Tomasi)

вход: $I_1, I_2; N$; окно W ; уровни L ; порог ε ; потолок $K_{\max}$

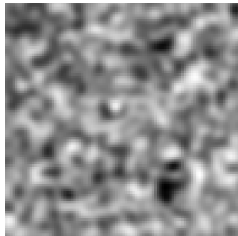
выход: $\{\mathbf{d}_i\}$ и маска «трек жив»

1. точки $p \leftarrow \text{Shi-Tomasi: } \min(\lambda_1, \lambda_2) > \tau$
2. **для** каждой точки p : $\mathbf{d} \leftarrow \mathbf{0}$, **старт** с нуля
 - 2.1 **для** $\ell = L, \dots, 0$ (с грубого уровня к полному): $I'_2(x) \leftarrow I_2(x + \mathbf{d}) - \text{варп}$
 - 2.1.1 **пока** $\|\Delta \mathbf{d}\| > \varepsilon$ и $k < K_{\max}$: $G, \mathbf{b}$ в W по (I_1, I'_2) ; $\Delta \mathbf{d} = -G^{-1} \mathbf{b}$; $\mathbf{d} \leftarrow \mathbf{d} + \Delta \mathbf{d}$; пересчитать I'_2
 - 2.1.2 **если** $\ell > 0$: $\mathbf{d} \leftarrow 2\mathbf{d}$ — в пикселях следующего уровня сдвиг вдвое больше
3. **туда-обратно:** трек $I_2 \rightarrow I_1$; **оставить**, если $e_{\text{FB}} < 1 \text{ px}$

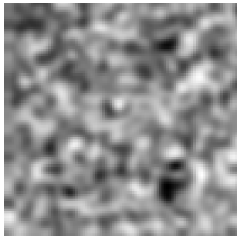
OpenCV: `goodFeaturesToTrack + calcOpticalFlowPyrLK (winSize=  $W$ , maxLevel=  $L$ , criteria=  $(\varepsilon, K_{\max})$ ).`

5. Собранный LK на паре кадров

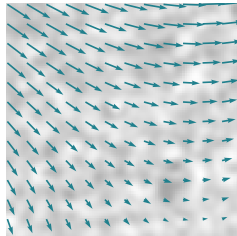
кадр t
синтетическая текстура



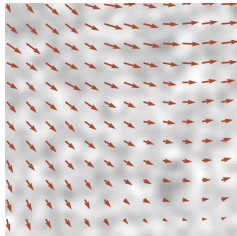
кадр $t+1$: сдвиг $(1.4, -0.6)$ px
+ поворот 1.2°



истинное поле
(стрелки $\times 4$)



оценка Lucas-Kanade, окно 15×15
EPE = 0.26 px



Сдвиг $(1.4, -0.6)$ px и поворот 1.2° , окно 15×15 , EPE ≈ 0.2 px. Если текстуры нет — окна мало: второе уравнение берут у соседей *по кадру*. Это Horn-Schunck. Долг про сдвиг 8 px отдадим пирамиде чуть позже.

6. Horn-Schunck: второе уравнение — гладкость поля

LK брал соседей в патче. HS (1981) берёт соседей по всему кадру: поток почти всюду гладкий. Это второе семейство ответов на то же OFC.

Энергия

$$E(u, v) = \iint (I_x u + I_y v + I_t)^2 dx dy \\ + \alpha^2 \iint (\|\nabla u\|^2 + \|\nabla v\|^2) dx dy.$$

Первый интеграл — data term (невязка OFC). Второй — regularizer: $\|\nabla u\|^2 = u_x^2 + u_y^2$. α — жёсткость «мембраны»: больше α — глаже поле, меньше доверия к шуму.

Сам интеграл не считаем — ищем его минимум. Для функционала условие минимума даёт Эйлер-Лагранж. Сначала общее напоминание, потом дифференцируем наш L .

6. Напоминание: Эйлер-Лагранж в 1D

Пусть задан функционал на гладких $f: [a, b] \rightarrow \mathbb{R}$:

$$J(f) = \int_a^b F(x, f(x), f'(x)) dx,$$

где f' — производная по x . Подынтегральная F дважды непрерывно дифференцируема. Её называют *функцией Лагранжа*, или лагранжианом.

Уравнение Эйлера-Лагранжа

Если J достигает экстремума на f , то

$$\frac{\partial F}{\partial f} - \frac{d}{dx} \left(\frac{\partial F}{\partial f'} \right) = 0.$$

Дальше ту же F пишем как L , неизвестную — как поле $u(x, y)$.

6. На плоскости — то же, плюс вторая координата

Поле $u(x, y)$, функционал $E[u] = \iint L(x, y, u, u_x, u_y) dx dy$. Две частные производные вместо одной y' :

Эйлер-Лагранж на плоскости

$$\frac{\partial L}{\partial u} - \frac{\partial}{\partial x} \left(\frac{\partial L}{\partial u_x} \right) - \frac{\partial}{\partial y} \left(\frac{\partial L}{\partial u_y} \right) = 0.$$

Два поля u и v — два таких уравнения. При варьировании u, v картинка I_x, I_y, I_t считается заданной (её мы уже посчитали с кадров).

Дальше выписываем плотность L из энергии HS и считаем все ∂L .

6. Плотность L : что именно оптимизируем

$E = \iint L \, dx \, dy$. Из двух интегралов энергии:

$$L = (I_x u + I_y v + I_t)^2 + \alpha^2 (u_x^2 + u_y^2 + v_x^2 + v_y^2).$$

Невязка OFC: $\rho = I_x u + I_y v + I_t$, тогда $L = \rho^2 + \alpha^2 (\|\nabla u\|^2 + \|\nabla v\|^2)$.

кусок L	за что штрафует
ρ^2	поток не объясняет смену яркости
$\alpha^2(u_x^2 + u_y^2)$	u скачет между соседями
$\alpha^2(v_x^2 + v_y^2)$	то же для v

α сидит только во втором куске: больше α — дороже излом поля.

6. Считаем производные L по u

$\rho = I_x u + I_y v + I_t$. Дифференцируем (множитель 2 потом сократится):

$$\frac{\partial L}{\partial u} = 2I_x \rho, \quad \frac{\partial L}{\partial u_x} = 2\alpha^2 u_x, \quad \frac{\partial L}{\partial u_y} = 2\alpha^2 u_y.$$

Вторые слагаемые Эйлера-Лагранжа — ещё раз по координате:

$$\frac{\partial}{\partial x} \frac{\partial L}{\partial u_x} + \frac{\partial}{\partial y} \frac{\partial L}{\partial u_y} = 2\alpha^2 (u_{xx} + u_{yy}) = 2\alpha^2 \Delta u.$$

EL для u (для v — с I_y)

$$2I_x \rho - 2\alpha^2 \Delta u = 0:$$

$$I_x(I_x u + I_y v + I_t) = \alpha^2 \Delta u, \quad I_y(I_x u + I_y v + I_t) = \alpha^2 \Delta v.$$

6. Дискретно: лапласиан — это я минус соседи

На сетке $\Delta u \approx \bar{u} - u$, где

$$\bar{u} = \frac{u(x-1, y) + u(x+1, y) + u(x, y-1) + u(x, y+1)}{4}.$$

Подставили в EL: $I_x \rho = \alpha^2(\bar{u} - u)$, $I_y \rho = \alpha^2(\bar{v} - v)$. Раскрыли ρ и перенесли неизвестные u, v влево:

$$\begin{pmatrix} \alpha^2 + I_x^2 & I_x I_y \\ I_x I_y & \alpha^2 + I_y^2 \end{pmatrix} \begin{pmatrix} u \\ v \end{pmatrix} = \begin{pmatrix} \alpha^2 \bar{u} - I_x I_t \\ \alpha^2 \bar{v} - I_y I_t \end{pmatrix}.$$

Правая часть содержит $\bar{u}, \bar{v}$ — средние *неизвестных* соседей. Пиксели связаны: это не N отдельных задач, а одна огромная система на весь кадр.

6. Почему итерации, а не одна формула

Закрытого ответа « u, v сразу на всём кадре» нет: чтобы узнать $u(x, y)$, нужно $\bar{u}$, а $\bar{u}$ — из u соседей, у тех — свои соседи, и так по всему изображению. Обращать матрицу размера $2HW \times 2HW$ дорого.

Якоби: притвориться, что соседи уже известны (берём их с шага k), решить дешёвую 2×2 в пикселе, потом повторить, пока поле не перестанет меняться.

Обозначим $D = \alpha^2 + I_x^2 + I_y^2$, $r^{(k)} = I_x \bar{u}^{(k)} + I_y \bar{v}^{(k)} + I_t$. При фиксированных соседях обратная матрица даёт

$$u^{(k+1)} = \bar{u}^{(k)} - \frac{I_x r^{(k)}}{D}, \quad v^{(k+1)} = \bar{v}^{(k)} - \frac{I_y r^{(k)}}{D}.$$

Старт $u^{(0)} = v^{(0)} = 0$: соседей ещё нет, есть только картинка. Каждый шаг подмешивает гладкость дальше от текстуры. Листинг — этот шаг в цикле.

6. Листинг: Horn-Schunck

Цикл — формула Якоби: 3.1 это $\bar{u}$, 3.2 это скобка r , 3.3–3.4 — обновление.

Алгоритм: Horn-Schunck

вход: кадры I_1, I_2 ; гладкость α ; итерации K

выход: плотный поток $u(x, y)$, $v(x, y)$

1. $I_x = \partial I / \partial x$, $I_y = \partial I / \partial y$, $I_t = I_2 - I_1$

2. $u^{(0)} = 0$, $v^{(0)} = 0$

3. **для** $k = 0, \dots, K - 1$, **для** каждого (x, y) :

3.1 среднее 4 соседей (то же для $\bar{v}^{(k)}$):

$$\bar{u}^{(k)} = \frac{u^{(k)}(x-1, y) + u^{(k)}(x+1, y) + u^{(k)}(x, y-1) + u^{(k)}(x, y+1)}{4}.$$

3.2 невязка OFC: $r^{(k)} = I_x \bar{u}^{(k)} + I_y \bar{v}^{(k)} + I_t$.

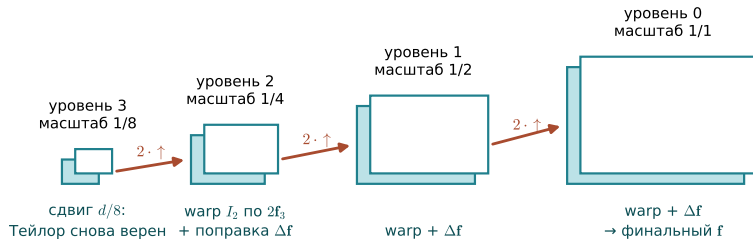
3.3–3.4 обновление, знаменатель общий:

$$u^{(k+1)} = \bar{u}^{(k)} - \frac{I_x r^{(k)}}{\alpha^2 + I_x^2 + I_y^2}, \quad v^{(k+1)} = \bar{v}^{(k)} - \frac{I_y r^{(k)}}{\alpha^2 + I_x^2 + I_y^2}.$$

4. вернуть $u^{(K)}$, $v^{(K)}$.

7. Тот же алгоритм по уровням

Сдвиг d пикселей на уровне 0 $\rightarrow d/2^L$ на уровне L : линейное приближение Тейлора снова законно



7. Листинг: coarse-to-fine

Обёртка. Внутри на каждом уровне — любой метод M из LK / HS / Farnebäck.

Алгоритм: пирамида

вход: I_1, I_2 ; число уровней L ; базовый метод M

выход: $\mathbf{f}$ на полном разрешении

1. пирамиды $I_1^{(\ell)}, I_2^{(\ell)}, \ell = 0, \dots, L$ (Гаусс + downsample $\times 2$)
2. $\mathbf{f} \leftarrow \mathbf{0}$ на уровне L
3. **для** $\ell = L, \dots, 0$:
 - 3.1 $I'_2 \leftarrow \text{warp}(I_2^{(\ell)}, \mathbf{f})$
 - 3.2 $\Delta \mathbf{f} \leftarrow M(I_1^{(\ell)}, I'_2)$ — поправка снова $\lesssim 1-2$ пх
 - 3.3 $\mathbf{f} \leftarrow \mathbf{f} + \Delta \mathbf{f}$; если $\ell > 0$: $\mathbf{f} \leftarrow 2 \cdot \text{upsample}(\mathbf{f})$

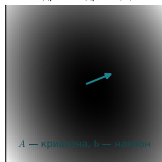
8. Farnebäck: плотный поток, не мембрана, а локальный полином

Локальный полином

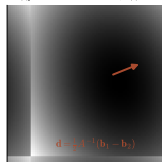
$$I(\mathbf{x}) \approx \mathbf{x}^T A \mathbf{x} + \mathbf{b}^T \mathbf{x} + c, \quad \mathbf{d} = \frac{1}{2} A^{-1} (\mathbf{b}_1 - \mathbf{b}_2).$$

A — кривизна 2×2 , $\mathbf{b}$ — наклон, c — уровень. Если $I_2(\mathbf{x}) = I_1(\mathbf{x} - \mathbf{d})$.

кадр t : квадрат A, b, c



кадр $t+1$: тот же патч, сдвиг d



10-20 мс на 640×480 , мажет границы. `cv2.calcOpticalFlowFarneback` (здесь разбираем один масштаб).

8. Листинг: Farneback

Алгоритм: Farneback

вход: кадры I_1, I_2 ; окно W ; веса w ; итерации N

выход: плотный поток $\mathbf{d}(x, y)$

1. в окрестности каждой точки оценить квадрат яркости:

$$I(\mathbf{x}) \approx \mathbf{x}^T A \mathbf{x} + \mathbf{b}^T \mathbf{x} + c \Rightarrow (A_1, b_1), (A_2, b_2), \quad A = \frac{1}{2}(A_1 + A_2).$$

2. при сдвиге $I_2(\mathbf{x}) = I_1(\mathbf{x} - \mathbf{d})$ линейные коэффициенты связаны так:

$$b_2 = b_1 - 2A\mathbf{d} \Rightarrow \mathbf{d} = \frac{1}{2}A^{-1}(b_1 - b_2).$$

3. одно уравнение $2A\mathbf{d} = b_1 - b_2$ неустойчиво. По окну W то же равенство в каждом p , взвешенный МНК:

$$\min_{\mathbf{d}} \sum_{p \in W} w(p) \|2A(p)\mathbf{d} - (b_1 - b_2)(p)\|^2.$$

Нормальные уравнения — как у LK, матрица G размера 2×2 :

$$G = \sum_{p \in W} w(p) (2A(p))^T (2A(p)), \quad h = \sum_{p \in W} w(p) (2A(p))^T (b_1 - b_2)(p),$$

$$G\mathbf{d} = h, \quad \mathbf{d} = G^{-1}h.$$

4. повторить N раз: сдвинуть коэффициенты I_2 на текущее $\mathbf{d}$, тем же МНК найти поправку $\Delta\mathbf{d}$,
 $\mathbf{d} \leftarrow \mathbf{d} + \Delta\mathbf{d}$.

8. Сводка: одно OFC, разные вторые допущения

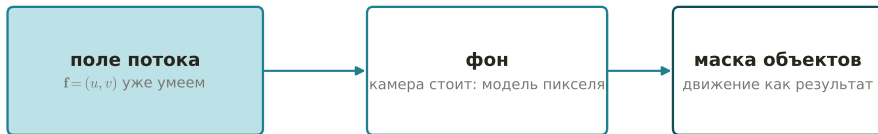
$$I_x u + I_y v + I_t = 0.$$

Метод	Тип	Сильные	Слабые	OpenCV
LK / KLT	sparse	быстро; точно на углах	только углы и текстура	PyrLK
Horn-Schunck	dense	поле везде; простая модель	мажет границы; медленно	—
Farnebäck	dense	плотный и быстрый на CPU	мажет границы; окклюзии	Farneback

Все закрывают нехватку уравнений разным допущением: окно (LK), соседи по кадру (HS), локальный квадрат (Farnebäck).

Классика потока закрыта. Дальше он — инструмент

Дальше поток — не цель, а инструмент



Камера стоит — значит, движение выдаёт объект. Строим модель «как пиксель выглядит обычно» — это фон.

9. Камера стоит: достаточно типичной яркости пикселя

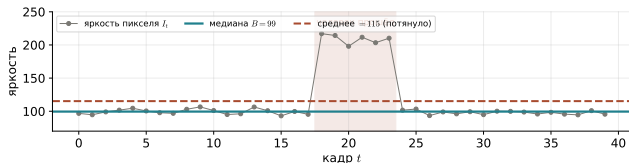
Не ищем соответствие кадров. Строим $B(x, y)$ — как этот пиксель выглядит без объектов.

Медианный фон

$$B = \text{median}_t I_t, \quad F_t = \mathbb{I}[|I_t - B| > \tau].$$

τ — порог в яркости; $F_t = 1$ — передний план.

Медиана, не среднее: редкий проход среднее потянет, медиану нет.



Плохо: вода, ветки, мерцающий экран — два «нормальных» цвета. Нужна смесь режимов.

9. Листинг: медианный фон

Алгоритм: медиана

вход: кадры $I_1, \dots, I_T$ при неподвижной камере; порог τ

выход: фон $B(x, y)$; маски переднего плана F_t

1. **для** каждого пикселя (x, y) : $B(x, y) \leftarrow \text{median}_t I_t(x, y)$
2. **для** каждого кадра t и пикселя: $F_t \leftarrow 1$, если $|I_t - B| > \tau$, иначе 0

Цикл по времени — только в медиане. Онлайн это не работает: нужен буфер.
GMM обновляет модель на каждом кадре.

9. Зачем моделировать фон смесью гауссиан?

Неподвижная камера: отделить объект от фона. Один эталон (медиана) держит *один* цвет пикселя.

У фона бывает несколько законных состояний: ветка / небо, вода, экран.

Идея GMM

Яркость пикселя — не число, а смесь K гауссиан. Каждая — свой режим фона.

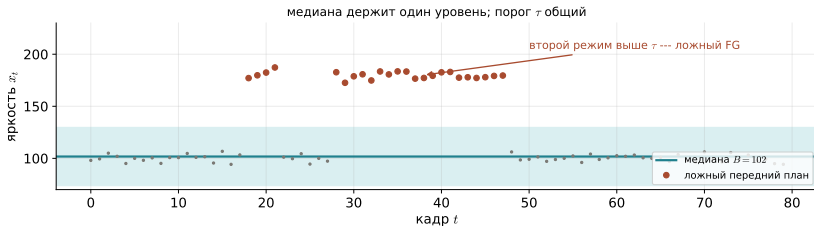


Две полки на $x(t)$ — два пика в гистограмме. Одному B этого не описать.

9. Ограничение медианного фона

Медиана даёт одно число. Частый режим остаётся фоном, редкий — «движением».

Нет σ : порог τ один на весь кадр. Нет онлайн-обновления: нужен буфер, скачок света ломает B . Нет забывания: устаревший режим и актуальный выглядят одинаково.



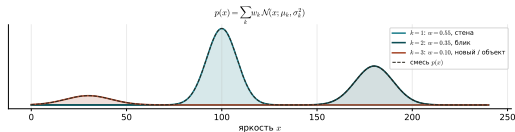
Второй пик выше τ — ложный передний план. Нужны несколько режимов и свой разброс.

9. Смесь гауссиан на пиксель

$K = 3\text{--}5$ компонент: вес w_k , среднее μ_k , разброс σ_k . Сортировка по w_k/σ_k : частые и узкие — фон, редкие — кандидаты в объект.

Новый кадр: совпало — обновить; нет — заменить слабейшую.

Экспоненциальное среднее: плавно забывает старое, выучивает новое.



9. Формулы: модель и проверка

Сначала описываем пиксель. Потом решаем: этот x_t уже известный режим?

1. Модель — чем заменяем одно число B

$$p(x) = \sum_{k=1}^K w_k \mathcal{N}(x; \mu_k, \sigma_k^2), \quad K = 3-5.$$

w_k — как часто режим встречался, μ_k — типичная яркость, σ_k — насколько «дрожит».

2. Проверка — совпало или нет

$$|x_t - \mu_k| < c\sigma_k, \quad c \approx 2.5.$$

Порог в единицах разброса: узкий режим (стена) строже, широкий (блик) терпимее. Цвет — тот же смысл, расстояние Махаланобиса:

$$D_M^2 = (\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}), \quad \text{совпало, если } D_M^2 < \tau.$$

9. Формулы: обучение и кто фон

Совпали — подмешиваем x_t в модель. Потом из K режимов выбираем фон.

3. Обучение — забыть старое, впитать новое

$M_k = 1$ только у совпавшей, у остальных 0:

$$w_k \leftarrow (1 - \alpha)w_k + \alpha M_k.$$

Совпавшую сдвигаем к x_t (скорость ρ , обычно порядка α):

$$\mu \leftarrow (1 - \rho)\mu + \rho x_t, \quad \sigma^2 \leftarrow (1 - \rho)\sigma^2 + \rho(x_t - \mu)^2.$$

Память $\approx 1/\alpha$: при $\alpha = 1/500$ — сотни кадров. Большое α быстрее ест свет — и стоящего человека.

4. Кто фон — не все K компонент

Сортировка по w_k/σ_k : частый и узкий устойчивее редкого широкого. Первые B , пока $\sum_{k=1}^B w_k > T \in [0.7, 0.9]$. Остальные — кандидаты в объект. $F_t = 1$, если x_t не попал в эти B .

9. Листинг: те же четыре действия в цикле

Алгоритм: Stauffer-Grimson

вход: поток I_t ; K ; α ; $c \approx 2.5$; порог фона T

выход: маска F_t ; на каждый пиксель $\{w_k, \mu_k, \sigma_k\}$

1. старт: первая гауссиана из кадра 1; остальные широкие, малый w
2. **для** кадра t , **для** пикселя x_t — проверка:
найти k с $|x_t - \mu_k| < c\sigma_k$ (цвет: $D_M^2 < \tau$)
3. **если** нашли — обучение совпавшей:
 $M = 1$ у неё, 0 у остальных; затем $w \leftarrow (1 - \alpha)w + \alpha M$, $\mu \leftarrow (1 - \rho)\mu + \rho x_t$,
 $\sigma^2 \leftarrow (1 - \rho)\sigma^2 + \rho(x_t - \mu)^2$
4. **иначе** новый режим вместо слабейшей:
 $w \leftarrow \alpha$, $\mu \leftarrow x_t$, $\sigma \leftarrow \sigma_0$
5. нормировать $\sum w = 1$; сортировать по w/σ
6. фон — первые B , пока $\sum w > T$; $F_t = 1$, если x_t не попал в эти B

Те же три кирпича — теперь обучаемые

Одно уравнение $I_x u + I_y v + I_t = 0$. Три способа добавить второе.

1. Окно

Lucas-Kanade

соседи в патче
едут одинаково

2. Соседи по кадру

Horn-Schunck / TV-L1

поле гладкое,
границу правит TV

3. Пирамида

coarse-to-fine

сдвиг $d/2^L$ снова
мал для Тейлора

Корреляция признаков $\leftrightarrow$ SAD / LK. Cost volume + warp $\leftrightarrow$ пирамида. Итерации GRU $\leftrightarrow$ Якоби в Horn-Schunck.

10. От ручного МНК к обучаемому сопоставлению

LK решал OFC в окне через МНК: все пиксели окна едут одинаково. Работает на углах, поле не плотное, окно и линейность яркости — вручную.

Что меняют FlowNetC и PWC-Net

Задача та же — найти соответствие двух кадров. Но: признаки вместо сырых яркостей, обучаемый декодер вместо МНК, плотный поток сразу во всех пикселях.

Скелет сохраняется: сначала похожесть окрестностей (аналог тензора G), из неё — сдвиг (u, v) .



10. FlowNetC: корреляционный слой — обучаемый аналог LK

Корреляция признаков

$$C(\mathbf{x}, \mathbf{d}) = \langle f_1(\mathbf{x}), f_2(\mathbf{x} + \mathbf{d}) \rangle, \quad \mathbf{d} \in [-20, 20]^2.$$

f_1, f_2 — выученные признаки кадров; $\mathbf{d}$ — проверяемый сдвиг. На точку — $41^2 = 1681$ гипотез, это столбец cost volume.

Сам слой весов не имеет — обучаются только признаки, которые в него подают. Диапазон ± 20 рх большой: считается один раз, без пирамиды.



10. Обучение FlowNetC: multi-scale EPE

Учим энкодер и декодер целиком, end-to-end: OFC нигде не записан. Данные — FlyingChairs: синтетика, поэтому истинный поток известен ТОЧНО.

Лосс

$$\mathcal{L}_{\text{EPE}} = \frac{1}{WH} \sum_{i,j} \|\mathbf{u}_{ij} - \hat{\mathbf{u}}_{ij}\|_2, \quad \mathcal{L} = \sum_s \lambda_s \mathcal{L}_{\text{EPE}}^{(s)}.$$

$\mathbf{u}_{ij}$ — истинный поток в пикселе, $\hat{\mathbf{u}}_{ij}$ — предсказанный; s — масштаб декодера, λ_s — его вес.

Штрафуются и промежуточные масштабы — сеть обязана давать разумный поток уже на грубых уровнях, а не только в конце.



декодер отдаёт поток на нескольких масштабах --- супервизия на каждом

10. PWC-Net: Pyramid + Warping + Cost volume

Название — три блока: пирамида признаков, warp признаков второго кадра текущей оценкой потока, локальный cost volume.

Почему окно ± 4 , а не ± 20

Warp заранее снимает найденную часть сдвига:

$$f_2^{\text{warp}}(\mathbf{x}) = f_2(\mathbf{x} + \mathbf{f}_{\text{up}}(\mathbf{x})), \quad C(\mathbf{x}, \mathbf{d}) = \frac{1}{C_l} \langle f_1(\mathbf{x}), f_2^{\text{warp}}(\mathbf{x} + \mathbf{d}) \rangle.$$

Остаток мал — хватает ± 4 пх. Дешевле FlowNetC по памяти и времени.

Идём от грубого уровня к тонкому — тот же coarse-to-fine, что у пирамиды LK.



10. Обучение PWC-Net: EPE по уровням пирамиды

Учим пирамиду признаков (общая для кадров), CNN-оценщик и context network. Warp и cost volume — операции без весов, как корреляция во FlowNetC.

Лосс

$$\mathcal{L} = \sum_l \alpha_l \mathcal{L}_{\text{EPE}}^{(l)}, \quad \mathcal{L}_{\text{EPE}}^{(l)} = \frac{1}{W_l H_l} \sum_{i,j} \|\mathbf{u}_{ij}^l - \hat{\mathbf{u}}_{ij}^l\|_2.$$

Каждый уровень l даёт свой поток; истину приводят к его разрешению $W_l \times H_l$. α_l — вес уровня.

Грубые уровни отвечают за большие смещения, тонкие — за точность на мелких структурах. Супервизия на каждом уровне, не только на выходе.

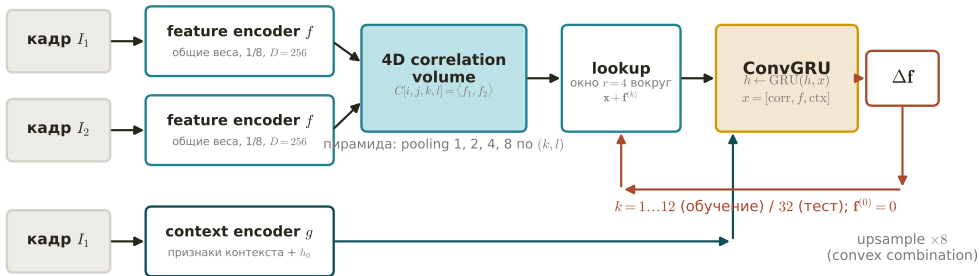
10. Что изменилось между FlowNetC и PWC-Net

	FlowNetC	PWC-Net
окно поиска аналог LK	± 20 px, один раз один большой «тензор»	± 4 px на каждом уровне пирамида маленьких + warp
обучается	энкодер + декодер	пирамида + оценщик + context
без весов данные лосс	корреляция FlyingChairs EPE по масштабам деко- дера	warp + cost volume + FlyingThings3D EPE по уровням пирамиды
стоимость	выше: полный диапазон	ниже: локальный поиск

PWC теряет мелкий быстрый объект на грубом уровне — та же слабость пирамиды. RAFT сравнивает все пары пикселей на одном разрешении.

10. RAFT: все пары + общие итерации

RAFT: все веса общие для всех итераций; лосс — сумма по итерациям с весами γ^{N-k}
 $\mathbf{f}^{(k+1)} = \mathbf{f}^{(k)} + \Delta \mathbf{f}$



$$C_{ijkl} = \langle \mathbf{g}_1(i, j), \mathbf{g}_2(k, l) \rangle.$$

Lookup вокруг $\mathbf{x} + \mathbf{f}^{(k)}$ — как KLT, но по признакам.

10. RAFT по шагам: два энкодера

Вход сети: I_1, I_2 — два RGB-кадра, каждый $(H, W, 3)$.

Шаг 1. Feature encoder — общие веса для обоих кадров

$I_1 (H, W, 3) \xrightarrow{\text{ResNet-подобная CNN}} \mathbf{g}_1 (H/8, W/8, 256)$, так же $I_2 \rightarrow \mathbf{g}_2$.

256 каналов — признаки для вычисления сходства: «что где выглядит похоже».

Шаг 2. Context encoder — отдельные веса, только для I_1

$I_1 (H, W, 3) \xrightarrow{\text{ResNet-подобная CNN}} \text{context} (H/8, W/8, 128+128)$.

128 каналов идут на инициализацию скрытого состояния GRU,

128 — на «подсказку» о содержимом сцены на каждой итерации.

Сходство и контекст — разные задачи, поэтому энкодера два: $\mathbf{g}$ отвечает «куда смотреть», context — «что это за сцена».

10. RAFT по шагам: correlation volume и пирамида

Шаг 3. Correlation layer — 4D-объём сразу по всем парам пикселей

вход: $\mathbf{g}_1, \mathbf{g}_2$ — оба $(H/8, W/8, 256)$

операция: $C(i, j, k, l) = \langle \mathbf{g}_1(i, j), \mathbf{g}_2(k, l) \rangle$ — для каждой пары точек

выход: C размерности $(H/8, W/8, H/8, W/8)$

Для каждого пикселя первого кадра храним похожесть со *всеми* пикселями второго, а не только с окрестностью.

Шаг 4. Correlation pyramid — пулинг по последним двум измерениям

вход: $C(H/8, W/8, H/8, W/8)$

операция: average pooling с ядрами 1, 2, 4, 8 по измерениям (k, l)

выход: 4 уровня пирамиды

Покрывают и маленькие, и большие смещения — без явного построения пирамиды изображений, как в PWC-Net.

Объём считается один раз до итераций; дальше из него только читают.

10. RAFT по шагам: старт и чтение из объёма

Шаг 5. Инициализация

$\mathbf{f}^{(0)} = \mathbf{0}$ ($H/8, W/8, 2$) — поток изначально нулевой везде;
 $h^{(0)}$ = tanh-часть context — скрытое состояние GRU из шага 2.

Дальше — цикл по итерациям $k = 0, \dots, N-1$ (обычно $N=12$ при обучении, 32 на инференсе).

Шаг 6 (итерация k). Lookup из correlation-пирамиды

вход: $\mathbf{f}^{(k)}$ ($H/8, W/8, 2$), correlation pyramid

операция: для каждой точки $\mathbf{x}$ берём предполагаемое соответствие $\mathbf{x}' = \mathbf{x} + \mathbf{f}^{(k)}(\mathbf{x})$ и билинейно извлекаем окрестность радиуса $r=4$ вокруг $\mathbf{x}'$ на каждом уровне пирамиды

выход: $\text{corr}^{(k)}$ ($H/8, W/8, 4(2r+1)^2$) — локальные корреляционные признаки

Шаг 7 (итерация k). Motion encoder

вход: $\text{corr}^{(k)}, \mathbf{f}^{(k)}$

операция: свёртки объединяют корреляцию и текущий поток в единый вектор движения

выход: $\text{motion}^{(k)}$ ($H/8, W/8, D_m$)

10. RAFT по шагам: обновление и полное разрешение

Шаг 8 (итерация k). ConvGRU update

вход: $h^{(k)}$, $\text{motion}^{(k)}$, context (128 каналов из шага 2)

операция: $h^{(k+1)} = \text{ConvGRU}(h^{(k)}, \text{motion}^{(k)}, \text{context})$

выход: $h^{(k+1)}$ ($H/8$, $W/8$, hidden)

Шаг 9 (итерация k). Flow head

вход: $h^{(k+1)}$

операция: небольшая свёрточная голова

выход: $\Delta \mathbf{f}^{(k)}$ ($H/8$, $W/8$, 2), $\mathbf{f}^{(k+1)} = \mathbf{f}^{(k)} + \Delta \mathbf{f}^{(k)}$

После N итераций цикл заканчивается — остаётся поднять разрешение.

Шаг 10. Convex upsampling — обучаемый

вход: $\mathbf{f}^{(N)}$ ($H/8$, $W/8$, 2), контекстные признаки

операция: взвешенное объединение соседних значений (а не просто билинейно)

выход: $\mathbf{f}$ (H , W , 2) — итоговое поле потока в полном разрешении

10. Листинг: RAFT

Алгоритм: RAFT

вход: кадры I_1, I_2 ; число итераций N (12 / 32)

выход: плотное $\mathbf{f}$

1. $\mathbf{g}_1, \mathbf{g}_2 \leftarrow$ feature encoder (общие веса, разрешение 1/8)
2. $\text{ctx}, h_0 \leftarrow$ context encoder(I_1)
3. $C_{ijkl} = \langle \mathbf{g}_1(i, j), \mathbf{g}_2(k, l) \rangle$; пирамида pooling по (k, l)
4. $\mathbf{f} \leftarrow \mathbf{0}, h \leftarrow h_0$
5. **для** $k = 1, \dots, N$:
 - 5.1 $\text{corr} \leftarrow \text{lookup}(C, \mathbf{x} + \mathbf{f}, r=4)$
 - 5.2 $h, \Delta \mathbf{f} \leftarrow \text{ConvGRU}(h, [\text{corr}, \mathbf{f}, \text{ctx}])$
 - 5.3 $\mathbf{f} \leftarrow \mathbf{f} + \Delta \mathbf{f}$
6. upsample $\times 8$

Те же роли, что у HS: сходство, соседи, итерации. Веса общие на все шаги.

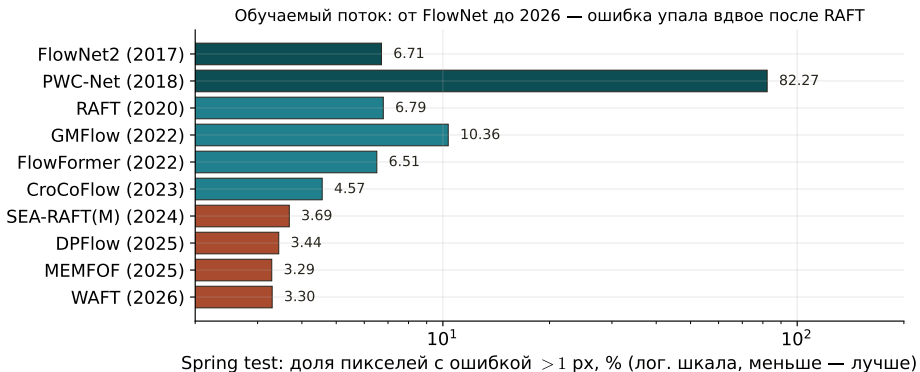
10. Новые имена, старые роли

Horn-Schunck	RAFT	Роль
data term из яркости	all-pairs correlation	сходство двух мест
гладкость α , лапласиан	свёртки GRU	уверенность к соседям
итерации Якоби	одни веса GRU, 12 / 32 раза	уточняем постепенно
ручной α	выученные веса	компромисс из данных

$$\mathcal{L} = \sum_{k=1}^N \gamma^{N-k} \|\mathbf{f}^{(k)} - \mathbf{f}^*\|_1.$$

Поздние оценки важнее; ранние уже должны быть разумными.
Старт $\mathbf{f}^{(0)} = \mathbf{0}$.

10. Что дали десять лет обучаемого потока



Итог — та же нить

1. Яркость + Тейлор $\rightarrow I_x u + I_y v + I_t = 0$: видна только $\mathbf{f}_n$
2. Окно (LK): $G = A^\top A$, трекаем углы, где оба λ велики
3. Соседи (HS / TV): $\text{data} + \lambda \cdot \text{smoothness}$; L2 мажет границу, TV режет
4. Пирамида: $d/2^L$; Farnebäck и DIS — CPU
5. Фон — GMM на пиксель: несколько режимов вместо одной медианы
6. FlowNet и PWC-Net: те же кирпичи слоями сети
7. RAFT — обучаемый HS: сходство, соседи, итерации — те же роли

Литература

- ▶ **Szeliski** — *Computer Vision*, гл. 9
- ▶ **Lucas, Kanade** 1981; **Horn, Schunck** 1981
- ▶ **Harris** 1988; **Shi, Tomasi** 1994
- ▶ **Farnebäck** 2003; **Zach, Pock, Bischof** — TV-L1, 2007
- ▶ **Teed, Deng** — RAFT, ECCV 2020
- ▶ **Stauffer, Grimson** 1999
- ▶ OpenCV Optical Flow; torchvision raft_small